Ordering, Not Duration: Timeout Interaction Hazards in Leaderless Task Claiming

Ordering, Not Duration: Timeout Interaction Hazards in Leaderless Task Claiming

Research companion to the Facetwork thesis (`thesis.md`). Empirical basis: the timeout, lease, reaper and cancellation machinery of a leaderless workflow runtime, and the production incidents that shaped each of them; all mechanisms, defaults and fixes cited are in the Facetwork repository and its fleet's execution history.

Abstract

A leaderless task queue has no coordinator to decide who is working on what. Ownership is instead asserted by an atomic claim and *maintained by the clock*: a lease that expires, a heartbeat that renews it, watchdogs that decide a claim has gone bad. Such a system therefore accumulates several independent timers — in ours, five — each introduced for a locally sound reason, each configurable, and each expressing an opinion about when a task stops belonging to its claimant. We argue that the correctness of this arrangement lies almost entirely in the **ordering** between those timers, not in their individual durations, and that the ordering is the part systems fail to make explicit. We catalogue nine interaction hazards observed in one production runtime, each with its signature, mechanism and mitigation: double execution from an inverted lease/execution-timeout relationship; a "zombie" claimant whose work is complete, wrong and invisible to itself; a timeout path that released work without consuming a retry and so re-claimed it forever; a timeout defeated by the blocking shutdown of the pool enforcing it; work that no timeout can interrupt at all, because the runtime cannot kill a thread inside a C extension; a safety guard that refuses the very operation its timer was meant to enable; a retry budget spent on deterministic failure; a circuit breaker that conflates a bad input with a bad handler; and an operational deploy timeout that silently pinned a fleet to a stale image. The through-line is that **every timeout in a leaderless system is a distributed agreement about current ownership**, and that agreements between independently-configured clocks should be *derived* rather than *configured*. We give the ordering invariants we now enforce, the two places our own configuration surface still lets an operator break them, and the instrumentation rule that would have surfaced most of these in an afternoon.

1. Introduction

Facetwork's runtime has no scheduler and no leader. A task is a document; a runner takes it with a single atomic compare-and-set; every other runner loses the race and moves on. This is the property that makes the fleet disposable — no runner is special, hosts may come and go — and it is described elsewhere in this corpus. What that design *displaces* rather than removes is the question every distributed executor must answer: **when does a claim stop being valid?**

With a coordinator, the answer is a policy in one place. Without one, the answer is a set of clocks that must agree. Ours grew to five: a task **lease**, a per-task **execution timeout**, a server **reaper**, a **stuck-task watchdog**, and a handler-declared **stage budget**. Each was added for a defensible local reason. None of them, individually, is interesting. Their *relationships* are where the failures live, and those relationships were mostly implicit until a failure made one explicit.

This paper is a catalogue and an argument. We contribute:

1. **A framing** (§3): in a leaderless claimer, a timeout is not a resource limit but an ownership assertion; correctness is an ordering property between timers, and duration is only a performance knob.
2. **Three ordering invariants** (§3) that our runtime now maintains, one of which is derived in code specifically so that an operator cannot accidentally invert it — and the two remaining places where the configuration surface still permits inversion (§7).
3. **A hazard catalogue** (§4): nine interactions, each with signature, mechanism and mitigation, drawn from production incidents rather than constructed examples.
4. **Prescriptions** (§5), including the instrumentation rule — *count committed transitions, not attempted rescues* — that would have shortened several of these investigations from months to an afternoon.

2. Background: five clocks

The claim. `claim_task` is one atomic `find_one_and_update`: a pending task becomes running and records the claiming `server_id`. There is no lock, no queue leader, and no fairness — just first-come-first-served per poll.

Lease (`FW_LEASE_DURATION_MS`). A claimed task carries `lease_expires`. When it passes, another runner may reclaim the task. This is the backstop for a claimant that has stopped making progress but has not been declared dead.

Execution timeout (`FW_TASK_EXECUTION_TIMEOUT_MS`, default 15 min). The *owning* runner's own bound on a single dispatch. On expiry it abandons the handler and returns the task to the pool.

Reaper (`FW_REAPER_TIMEOUT_MS`, default 2 min). Server-level, not task-level: if a runner's heartbeat goes stale the reaper resets *its* tasks to pending. This is the fast path for a genuinely dead host, and it is deliberately independent of the lease.

Stuck-task watchdog (`FW_STUCK_TIMEOUT_MS`, default 30 min). Task-level inactivity: a task whose last heartbeat is older than its timeout is reclaimed even though its server is alive.

Stage budget. A handler may declare that a named phase will legitimately take much longer than any of the above (`ctx.stage("pbf_scan", timeout_ms=30*60_000)`). It suppresses each clock in the way appropriate to that clock: the stuck-task watchdog skips the task outright while the budget is live; the owning runner's execution timeout fires only once *both* its own deadline and the stage deadline have passed; and the lease is renewed to *cover* the budget, `lease_expires = max(now + lease_ms, budget_expires)`. Three different suppressions, because they are three different assertions.

Heartbeat. The renewal signal underneath all of it, and — importantly for §4.2 — a *write*, which means it can be gated on still owning the task.

3. The invariant: who owns this work now

Every one of those five timers, when it fires, changes the answer to a single question: *which runner, if any, is entitled to finish this task and write its result?* Once framed that way, the failure modes stop looking like unrelated bugs and start looking like disagreements.

Three orderings must hold.

I1 — The owner's own bound must fire before anyone else's reclaim. If another runner can reclaim a task while its current claimant is still executing, the work runs twice. So the lease must outlast the execution timeout. We do not ask an operator to arrange this; the store *derives* it:

```
lease_ms = max(DEFAULT_LEASE_MS, execution_timeout + LEASE_OVER_EXEC_GRACE_MS)
         = max(5 min, 15 min + 1 min) = 16 min
```

The one-minute grace is the whole point: it guarantees the owning runner's watchdog resets a wedged task *before* the lease makes it available to a stranger. The 5-minute floor is a floor, not the operative value — a reader skimming the defaults table would reasonably but wrongly conclude the lease is five minutes.

The same ordering binds every *other* path that can take a running task away from its claimant, and stating I1 in terms of the lease alone was too narrow. There are three such paths, and two of them were unguarded: the per-task stuck reap compared against the very same `timeout_ms` the owning runner dispatches on — so the reaper and the owner fired *together*, a coin-flip over whether the task was handed to a second runner while the first was winding down — and the default stuck reap used an independently-configured threshold with no relation to the execution timeout at all. The stock defaults (30 min stuck, 15 min execution) happen to be ordered correctly, which is why this went unnoticed; a deployment raising the execution timeout to four hours without raising the stuck timeout is not, and ours had raised it. The generalised invariant is therefore: **no external reclaim path may fire before the owner's own bound plus a grace**, and the grace is now one named constant shared by all three.

I2 — Server death must be recovered faster than task slowness. A dead host should not hold work for a lease period. The reaper therefore runs on heartbeat staleness (2 min) and is independent of the lease (16 min): a *slow* task is protected by I1, a *dead server's* tasks are not. Collapsing these two into one timer — the intuitive simplification — makes one of the two cases wrong whichever value is chosen.

I3 — A declared long phase must suppress every clock that would otherwise fire during it, including the one that grants ownership elsewhere. It is not enough for the stage budget to silence the watchdog; if it did only that, the lease would still expire mid-phase and hand the task to another runner. Hence the lease renewal takes the *maximum* of the ordinary lease and the declared budget. A budget that suppressed the watchdog but not the lease would convert "this handler legitimately takes 40 minutes" into "this handler is executed twice, 16 minutes apart".

4. Hazard catalogue

4.1 Double execution by lease/execution-timeout inversion

Signature. A long handler completes; its results are already produced by a second runner. Output is correct-looking but the work was paid for twice — in an external API's rate limit, or a PostGIS import, corrupt. **Mechanism.** I1 inverted: lease shorter than the execution bound. **Mitigation.** Derivation, as above, rather than two independent knobs — and, since the configuration surface still exposed the inversion, a **clamp**: an explicit `FW_LEASE_DURATION_MS` below the derived floor is raised to it, with a warning naming both values and the consequence. The clamp recomputes the floor from the *current* execution timeout on every call, so it also closes the per-domain variant (a domain raising its own execution timeout past a previously-fine explicit lease). The warning fires once per process: the function runs on every claim, heartbeat and stage renewal, so an unthrottled warning would flood the logs of exactly the host being diagnosed. **Note on the choice.** Clamping silently overrides an operator's explicit configuration, which is normally poor manners; it is justified here because the setting is not a preference but an invariant whose violation is invisible (duplicated work looks like success) and whose blast radius is data corruption. Refusing to start would be the louder alternative, but the lease is read per-operation rather than at startup, so a raise would fail a running fleet rather than a misconfiguration.

4.2 The zombie: a claimant that is wrong and cannot tell

Signature. A handler runs to completion and its result vanishes; logs show a terminal write "dropped". Worse, before the fix, a reclaimed handler's heartbeat kept renewing the *new* owner's lease. **Mechanism.** Reclaim (by reaper or lease) transfers ownership while the original process is still running. That process's view of the world is stale in a way it cannot detect locally: it holds a task object that says it is the owner. **Mitigation.** Ownership gating on *both* writes. The heartbeat renewal carries `expected_server_id` and is a no-op once the row moved, so a zombie cannot extend a successor's lease; terminal writes go through an ownership-gated save that drops the write and logs it rather than clobbering the new claimant's state. Later, cooperative cancellation gave the zombie a way to *notice*: the reclaim is one of the conditions its cancellation token reports, so a checking handler stops instead of finishing work nobody will accept. **Generalisation.** In a leaderless claimer, "am I still the owner?" is not answerable from local state and must be a *gated write* or an explicit query.

4.3 The retry-less timeout release

Signature. A handler that reliably exceeds its timeout is re-claimed immediately, forever, by a fleet that never dead-letters it. **Mechanism.** The timeout path reset the task to pending without incrementing the retry count or setting a backoff. Every other failure path routed through the shared retry contract; this one predated it. The task was therefore *eligible again* the instant it was released, and nothing counted the attempts. **Mitigation.** Route the timeout through the same retry-then-dead-letter transition as every other failure, with exponential backoff. The lesson is not about timeouts specifically: **a state transition that returns work to the pool must always pass through the accounting that decides when to stop.**

4.4 The timeout defeated by the pool enforcing it

Signature. A per-handler timeout is configured, expires, and the runner still blocks for the handler's full real duration. **Mechanism.** The bounded dispatch ran in a thread pool used as a context manager. On timeout the code left the `with` block — whose implicit `shutdown(wait=True)` blocked until the runaway handler returned. The timeout fired and then waited for exactly the thing it was bounding. **Mitigation.** `shutdown(wait=False)`: abandon the wedged thread, free the poll slot immediately, and let the global execution timeout be the backstop. **Generalisation.** A timeout is only as strong as the weakest *join* on its path; enforcing a bound while holding a blocking handle on the bounded work is a contradiction that reads as correct code.

4.5 Work that cannot be interrupted at all

Signature. Cancellation "works" — the task is marked cancelled — and the handler keeps consuming CPU, disk and API quota for another hour. **Mechanism.** Python cannot safely kill a thread blocked in a C extension. `Future.cancel()` cannot interrupt a running handler; it only prevents a not-yet-started one. No timeout the runtime owns can stop a `pyosmium` `apply_file()` mid-scan. **Mitigation.** Cooperative cancellation: the runtime injects a token, the handler checks it at a boundary of its choosing, and non-cooperative handlers remain bounded — loosely — by the execution timeout. Verified on a 1 GB scan: a 28.6 s traversal aborted at 20.0 s, the exception unwinding out of the C-level call. **The interaction this creates.** The check is time-gated (10 s in the first adopter), so *worst-case stop latency is one check interval*, and a unit of work shorter than the interval is never checked at all. That is benign — such work finishes anyway — but it means cancellation cannot be demonstrated on a small input, a fact that made our first acceptance test report a false failure.

4.6 A guard that refuses the operation its timer enables

Signature. `terminate-workflow` on a live, healthy long-running run is **refused**: "*1 task(s) heartbeating within the last 30s — run with --force*". **Mechanism.** A sensible guard against terminating live work, keyed on recent heartbeat. But a healthy long handler heartbeats *by design* — that is how it survives §2's watchdogs. So the guard triggers precisely in the case cancellation exists to serve. **Mitigation.** None needed in code; the guard is correct. The hazard is documentation: cancelling live work is *always* the `--force` path, and an operator meeting the refusal for the first time reasonably reads it as a failure of the feature. **Generalisation.** Liveness signals get reused as safety signals, and the two readings can invert: "still heartbeating" means *healthy* to a watchdog and *don't touch* to a guard.

4.7 A retry budget spent on a deterministic failure

Signature. A fan-out burns 150 tasks × 5 attempts, with exponential backoff between each, to reach the answer it had on the first attempt. **Mechanism.** The retry contract assumes transience, and nothing in an exception distinguishes "the database restarted" from "this region has no motorway tier". Absent a signal, every failure is optimistically retried. **Mitigation.** A non-retryable error contract (`PermanentError`) that dead-letters immediately without consuming the budget, while still failing the step so error propagation runs unchanged. **Interaction with §4.5.** These are the two halves of one idea — *stop doing work whose result nobody will accept* — arrived at from opposite directions: cancellation is the operator telling the handler, `PermanentError` is the handler telling the runtime.

4.8 Health conflated with input validity

Signature. A handler is taken out of service for every item in a fan-out because a few items had bad input. **Mechanism.** A per-facet circuit breaker counts consecutive failures and stops claiming. Timeouts and exceptions correctly count toward it — a handler that keeps failing *is* unhealthy. But a deterministic input rejection is a fact about the *item*, and counting it lets five bad rows in a 3,000-row fan-out close the breaker for the other 2,995. **Mitigation.** Permanent errors are excluded from the breaker by construction, with a contrast test asserting the breaker still opens for genuinely failing handlers. **Every failure counter needs an explicit answer to "a failure of what?"**

4.9 The operational timeout: a deploy that silently did nothing

Signature. A rollout reports success — image built, pushed, central config version bumped — and the fleet keeps running the old code. The agent log shows `start failed (exit 124)`. **Mechanism.** The reconcile shells out to `docker compose up` under a 600-second bound. A fresh multi-gigabyte image pull exceeds it, so the reconcile is killed mid-pull, having changed nothing, and retries into the same wall on the next cycle. 124 is the shell's timeout code, not the application's. **Mitigation.** Pre-pull the image outside the bound, then reconcile — the pull is then a cache hit well inside 600 s. **Why it belongs in this catalogue.** The same class as §4.4: a bound applied to an operation whose duration is data-dependent and unbounded in principle. Deployment timeouts are ownership assertions too — here, about which image version the fleet is entitled to be running — and the failure is the same shape: the system reports the *intent* (config bumped) while the *effect* never landed.

5. Prescriptions

P1 — Derive relationships; configure only leaves. Any two timers with a required ordering should not both be free knobs. Our lease is computed from the execution timeout plus an explicit grace, and the grace is a named constant so the invariant is greppable.

P2 — Gate by ownership, not by time. Time tells a claimant when it *might* have lost the task; only a conditional write tells it whether it *has*. Every write that finalises work should carry the claim identity, and the drop should be logged loudly enough to appear in an incident search.

P3 — Make "legitimately long" an explicit, first-class state. The stage budget exists so that a 40-minute phase is *declared* rather than inferred from silence. Without it, the only way to support long work is to raise every global timeout, which degrades detection for everything else.

P4 — Cooperative cancellation is the only kind that exists. Design for a handler that checks, bound the ones that do not with a coarse timeout, and document the check interval as the stop-latency it is.

P5 — Every return-to-pool goes through the accounting. §4.3's bug was a path that skipped the retry counter. A single transition function, used by every failure path, is worth more than five correct call sites.

P6 — Instrument effect, not attempt. This is the rule with the highest yield in this corpus. Several investigations here — and the two in the companion liveness paper — were prolonged because the logs reported *what the loop tried* ("resumed 68 steps", "applying config v146") while the *committed effect* was nothing. A recovery loop should count state transitions it committed, and a deploy should verify the running artefact by content rather than by tag.

6. Related work

Temporal separates *schedule-to-start*, *start-to-close* and *heartbeat* timeouts per activity, which is precisely the acknowledgement that one timer cannot express these different assertions; its heartbeat timeout is the direct analogue of our watchdog, and its retry policy the analogue of §4.3's accounting. The difference is architectural rather than conceptual: Temporal's service arbitrates the timers centrally, so ordering is enforced by one implementation, whereas a leaderless design distributes that responsibility to every runner and every store backend — which is why our invariants must be derived in shared code and asserted by parity tests across two persistence implementations.

Celery's *visibility timeout* on a broker without acknowledgement semantics is the canonical instance of §4.1: a task redelivered while still executing. SQS's visibility timeout with explicit extension is the same shape as our stage budget, and its documentation carries the same warning about duplicate delivery. Kubernetes liveness probes exhibit §4.6 in a different guise — a probe that restarts a container that is merely slow — and the standard remedy (a separate startup probe) is P3: make "legitimately long" a distinct state rather than a longer threshold. Chubby's and ZooKeeper's session leases give the classic treatment of lease-versus-clock skew, which we sidestep rather than solve: our leases are compared against a single database's clock, so we inherit its ordering and do not attempt fencing tokens.

The specific contribution here is neither a new mechanism nor a proof, but the *interaction* framing: each hazard above is individually understood in the literature, while all nine arose in one system from combining locally-correct timers without an explicit ordering discipline.

7. Threats to validity and scope

This is one runtime, one workload family, and one small fleet; the catalogue is what we encountered, not a survey, and there is no claim that these nine are exhaustive or representative in frequency. Several entries are single incidents whose mitigation has held rather than controlled experiments.

Both configuration-surface hazards named in the first draft of this paper were closed while it was being written, which is worth recording as a datapoint about the exercise rather than hidden: writing the catalogue is what made the two live inversions legible as *the same* invariant, and the fix was nine lines. An explicit `FW_LEASE_DURATION_MS` below the derived floor is now clamped up with a one-time warning, and because the floor is recomputed per call from the current execution timeout, the per-domain variant — a domain author raising an execution timeout past a previously-adequate lease — is closed by the same code. Nine tests cover it, four of which were confirmed to fail against the pre-clamp implementation.

The item this section listed as open in its first draft — that the reaper and stuck-task timeouts had no articulated relationship, and that "an invariant nobody has articulated is not enforced, it is merely untested" — was then articulated and enforced (§3). Writing it down was what exposed it: both stuck reaps could fire at or before the owning runner's own deadline, one of them because it compared against the identical number the owner dispatches on. This is the second time in this paper's short history that stating an invariant plainly revealed it to be violated in the very system being described, which we record as the method's main practical argument rather than as a coincidence.

What remains genuinely open is the *reaper* (server-liveness) side. Its 2-minute threshold is deliberately independent of the task-level clocks — that is I2 — but "independent" is a claim about intent, and we have not derived a bound on how far it may drift toward them before a slow-but-alive host has its work stolen. We know of no violation; we also have no test that would notice one.

The cancellation stop-latency claim (§4.5) is measured on a single scan workload at one check interval; the acceptance evidence for the surrounding liveness fix is a 60-iteration run, not the 3,167-iteration scale at which the original failures occurred. Finally, we have not attempted the adversarial testing this area invites — deterministic simulation with an aggressive clock, in the style of FoundationDB or TigerBeetle, would likely find hazards this catalogue misses, and is the obvious next step.

8. Conclusion

A leaderless claimer buys its central property — that no runner is special — by giving up a coordinator's ability to answer "who owns this work?" in one place. The answer is reconstituted from clocks, and the clocks accumulate: five here, each locally justified, each configurable, each with an opinion. We found that essentially none of the failures were about a *duration* being wrong. They were about two timers disagreeing (§4.1, §4.5), a timer being enforced by something that waited for what it bounded (§4.4, §4.9), a transition that skipped the accounting (§4.3), a claimant with no way to learn it had been superseded (§4.2), or a counter that could not say what it was counting (§4.8).

The discipline that follows is small and unglamorous: derive the relationships that must hold, gate the writes that finalise work on the claim that produced them, make long work declare itself, and instrument what actually got committed. The last of those is the one we keep relearning — across this paper and its companion, the most expensive investigations were the ones where every log said the system was doing its job.